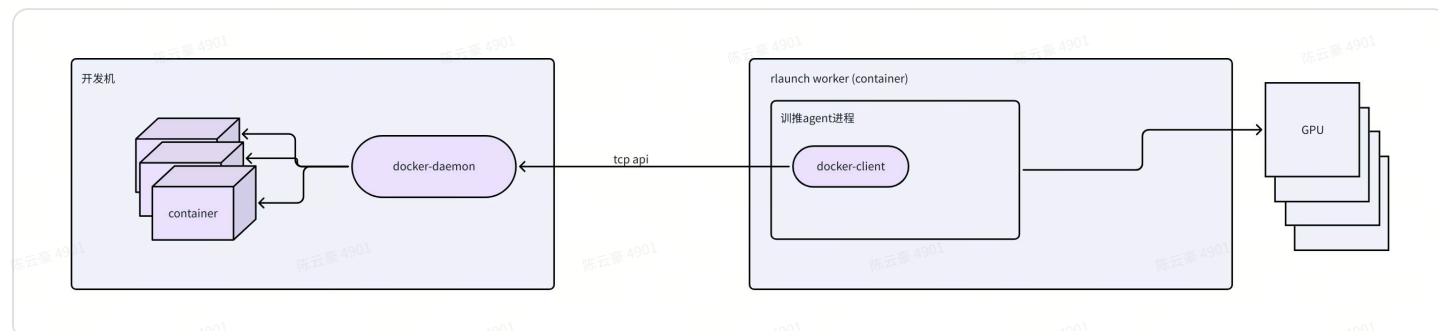


rlaunch worker支持docker的方法

基本原理



通过docker daemon remote api允许rlaunch worker访问开发机本地的docker daemon，当rlaunch worker内希望调用docker时，直接调用开发机docker daemon。

参考的官方文档：

<https://docs.docker.com/engine/daemon/remote-access/>

Configure remote access for Docker daemon

Configuring remote access allows Docker to accept requests from remote hosts by configuring it to listen on an IP address and port as well as the Unix socket

操作步骤

1. 在开发机中配置docker服务启动参数，允许外部tcp控制。

Code block

```
1 sudo vim /lib/systemd/system/docker.service
2
3 增加下图下划线的tcp端口配置，端口可以自定义
4 -H tcp://0.0.0.0:2376
```

```
[Unit]
Description=Docker Application Container Engine
Documentation=https://docs.docker.com
After=network-online.target firewallld.service containerd.service
Wants=network-online.target
Requires=docker.socket
Wants=containerd.service

[Service]
Type=notify
# the default is not to use systemd for cgroups because the delegate issues still
# exists and systemd currently does not support the cgroup feature set required
# for containers run by docker
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://0.0.0.0:2376 --containerd=/run/containerd/containerd.sock
ExecReload=/bin/kill -s HUP $MAINPID
TimeoutSec=0
RestartSec=2
Restart=always
# Note that StartLimit* options were moved from "Service" to "Unit" in systemd 229.
# Both the old, and new location are accepted by systemd 229 and up, so using the old location
# to make them work for either version of systemd.
StartLimitBurst=3
# Note that StartLimitInterval was renamed to StartLimitIntervalSec in systemd 230.
# Both the old, and new name are accepted by systemd 230 and up, so using the old name to make
# this option work for either version of systemd.
StartLimitInterval=60s
# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNOFILE=infinity
LimitNPROC=infinity
LimitCORE=infinity
# Comment TasksMax if your systemd version does not support it.
# Only systemd 226 and above support this option.
TasksMax=infinity
# set delegate yes so that systemd does not reset the cgroups of docker containers
Delegate=yes
# kill only the docker process, not all processes in the cgroup
KillMode=process
OOMScoreAdjust=-500

[Install]
WantedBy=multi-user.target
~
~
```

2. 开发机docker服务重启（注意，这一步已运行中容器会被停掉，所以需要确保容器内数据已保存）

Code block

- 1 sudo systemctl daemon-reload
- 2 sudo systemctl restart docker

3. 确认tcp接口可用

Code block

- 1 # 改成你自己配置的端口，检查是否监听
- 2 sudo ss -nltpt | grep 2376

```
chenhaodong@chd-test-docker:~$ ss -nltpt | grep 2376
tcp        LISTEN    0          0            *:2376          *:*
```

```
1 # 使用docker客户端验证tcp接口可以走通
2 docker --host tcp://0.0.0.0:2376 images
3
```

```
chenhaodong@chd-test-docker:~$ docker --host tcp://0.0.0.0:2376 images
REPOSITORY                                TAG                IMAGE ID           CREATED            SIZE
registry.h.pjlab.org.cn/ailab-zskj/chenhaodong-dev  ubuntu22.04       bd0fe733401a      2 weeks ago      3.71GB
registry.h.pjlab.org.cn/ailab-zskj/ml-base         22.04-huspy-v1    84de8bed49ac      6 weeks ago      3.7GB
chenhaodong@chd-test-docker:~$
```

4. 查看并记录自己开发机的IP

Code block

```
1 ip addr list
2 # 获取eth0网卡的IP, 100.开头的
```

```
chenhaodong@chd-test-docker:~$ ip addr list
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9000 qdisc pfifo_fast state UP group default qlen 1000
   link/ether 88:66:64:64:40:67 brd ff:ff:ff:ff:ff:ff
   altname enp0s8
   inet 100.100.123.123/24 brd 100.100.64.255 scope global eth0
       valid_lft forever preferred_lft forever
   inet6 fe80::8a66:64ff:fe64:4067/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 02:42:15:a1:a9:40 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
   inet6 fe80::42:15ff:fe01:a940/64 scope link
       valid_lft forever preferred_lft forever
```

5. 接下来到了你启动自己rlaunch worker的时候了, 使用rlaunch命令创建worker

Code block

```
1 # 我拿一个我自己的命令做演示, 这里替换成你自己的启动命令
2 # 请注意增加一个DOCKER_HOST=tcp://100.x.x.x的环境变量, 这里的IP填的是你开发机的IP
3 rlaunch --gpu=8 --memory=800000 --charged-group=chd_compute_gpu --cpu=50 --
  image=registry.h.pjlab.org.cn/ailab-puyu-puyu_gpu/lmdeploy:v0.10.0-cu12.8 --
  mount=gpfs://gpfs1/chenhaodong:/mnt/shared-storage-user/chenhaodong -e
  DOCKER_HOST=tcp://100.100.123.123:2376 --positive-tags node/gpu-lg-cmc-h-h200-
  0685.host.h.pjlab.org.cn --private-machine=group -- bash
```

6. 进入容器后测试

Code block

```
1 # 我是用的是Python的docker库做的测试, 测试脚本如下
2
3 import docker
```

```

4
5 def test_docker_connection():
6     try:
7         client = docker.from_env()
8         # 验证 daemon 可连通
9         client.ping()
10        print("Y Docker daemon reachable")
11        # 登录H集群镜像仓库
12        client.login(username=<AK>, password=<SK>,
13        registry="registry.h.pjlab.org.cn")
14        # 拉取并运行一个轻量镜像
15        container = client.containers.run(
16            "registry.h.pjlab.org.cn/ailab/python:3.10-ubuntu22.04",
17            "echo hello from docker",
18            remove=True
19        )
20        print(container.decode("utf-8"))
21        print("Y Docker container ran successfully")
22
23    except docker.errors.APIError as e:
24        print(f"X Docker API error: {e}")
25    except Exception as e:
26        print(f"X General error: {e}")
27
28 if __name__ == "__main__":
29     test_docker_connection()

```

```

root@chd-test-docker-m5h9b-96975-worker-0:/# cat > /tmp/test_docker.py << "EOF"
import docker

def test_docker_connection():
    try:
        client = docker.from_env()
        # 验证 daemon 可连通
        client.ping()
        print("Y Docker daemon reachable")
        # 登录H集群镜像仓库
        client.login(username="d519fba8a05e73fe8827dfdc3bcddef1", password="ce[REDACTED]25", registry="registry.h.pjlab.org.cn")
        # 拉取并运行一个轻量镜像
        container = client.containers.run(
            "registry.h.pjlab.org.cn/ailab/python:3.10-ubuntu22.04",
            "echo hello from docker",
            remove=True
        )
        print(container.decode("utf-8"))
        print("Y Docker container ran successfully")

    except docker.errors.APIError as e:
        print(f"X Docker API error: {e}")
    except Exception as e:
        print(f"X General error: {e}")

if __name__ == "__main__":
    test_docker_connection()
EOF
root@chd-test-docker-m5h9b-96975-worker-0:/# python /tmp/test_docker.py
Y Docker daemon reachable
hello from docker

Y Docker container ran successfully
root@chd-test-docker-m5h9b-96975-worker-0:/#

```